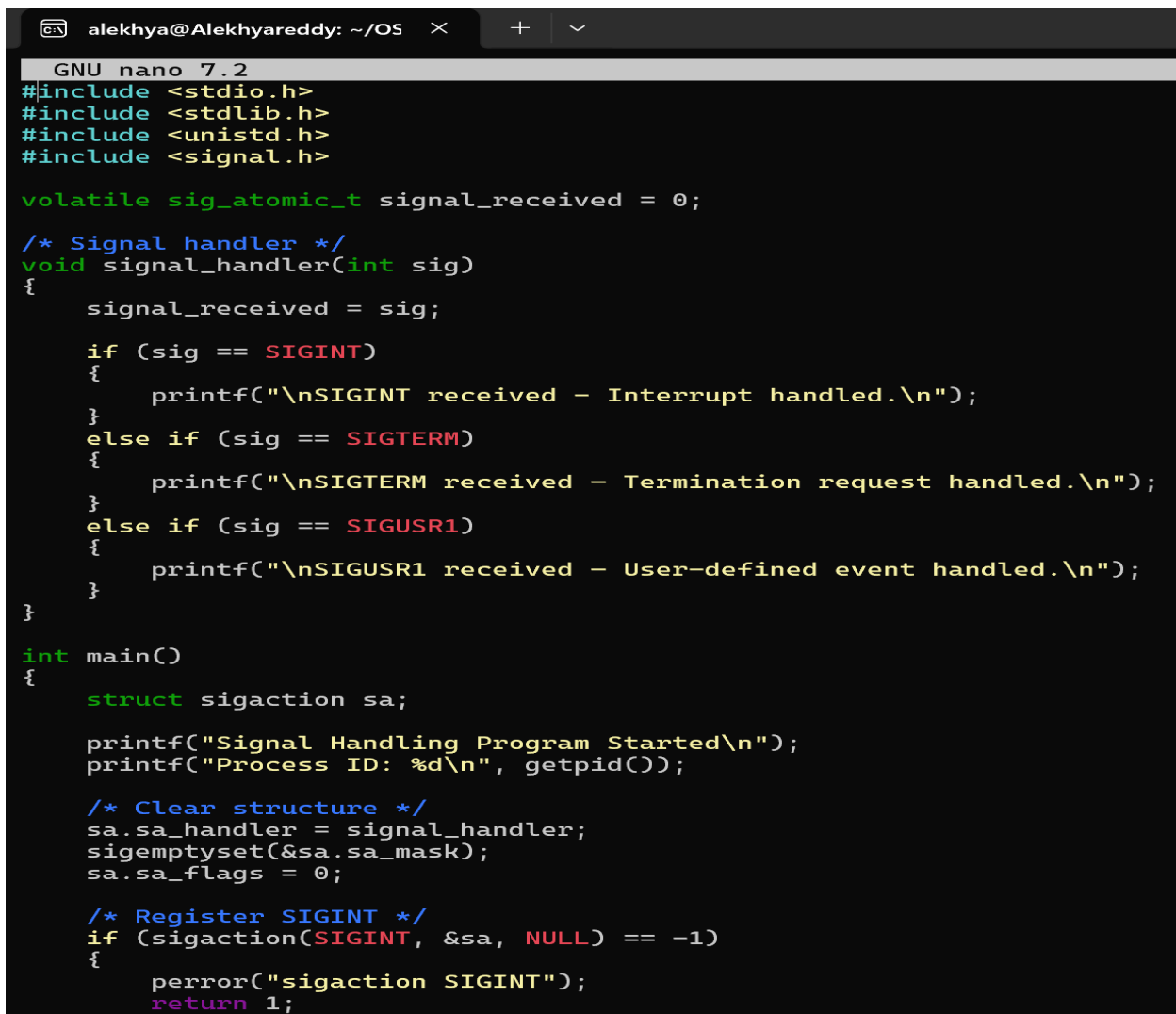


SKILL WEEK-10

1 . To Register Handlers, Configure Signal Actions, Handle Interrupts, Manage Signal Context, Verify Stability, Test Signal Events

Aim : To register signal handlers, configure signal actions, handle interrupts, manage signal context, verify program stability, and test different signal events.

Program :



```
GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>

volatile sig_atomic_t signal_received = 0;

/* Signal handler */
void signal_handler(int sig)
{
    signal_received = sig;

    if (sig == SIGINT)
    {
        printf("\nSIGINT received - Interrupt handled.\n");
    }
    else if (sig == SIGTERM)
    {
        printf("\nSIGTERM received - Termination request handled.\n");
    }
    else if (sig == SIGUSR1)
    {
        printf("\nSIGUSR1 received - User-defined event handled.\n");
    }
}

int main()
{
    struct sigaction sa;

    printf("Signal Handling Program Started\n");
    printf("Process ID: %d\n", getpid());

    /* Clear structure */
    sa.sa_handler = signal_handler;
    sigemptyset(&sa.sa_mask);
    sa.sa_flags = 0;

    /* Register SIGINT */
    if (sigaction(SIGINT, &sa, NULL) == -1)
    {
        perror("sigaction SIGINT");
        return 1;
    }
}
```

```

}

/* Register SIGTERM */
if (sigaction(SIGTERM, &sa, NULL) == -1)
{
    perror("sigaction SIGTERM");
    return 1;
}

/* Register SIGUSR1 */
if (sigaction(SIGUSR1, &sa, NULL) == -1)
{
    perror("sigaction SIGUSR1");
    return 1;
}

printf("\nSignal handlers registered successfully.\n");
printf("Waiting for signal events...\n");
printf("Press Ctrl+C to test SIGINT.\n");

while (1)
{
    pause();

    if (signal_received != 0)
    {
        printf("Program is still running and stable.\n");
        signal_received = 0;
    }
}

return 0;
}

```

Output :

Terminal-1 :

```

alekhya@Alekhyareddy: ~/OS  ×  +  v
alekhya@Alekhyareddy:~$ cd ~/OSSP
alekhya@Alekhyareddy:~/OSSP$ nano task_signal_actions.c
alekhya@Alekhyareddy:~/OSSP$ gcc -Wall -Wextra task_signal_actions.c -o task_signal_actions
alekhya@Alekhyareddy:~/OSSP$ ./task_signal_actions
Signal Handling Program Started
Process ID: 488

Signal handlers registered successfully.
Waiting for signal events...
Press Ctrl+C to test SIGINT.
^C
SIGINT received - Interrupt handled.
Program is still running and stable.

SIGUSR1 received - User-defined event handled.
Program is still running and stable.

SIGTERM received - Termination request handled.
Program is still running and stable.

SIGINT received - Interrupt handled.
Program is still running and stable.
Killed
alekhya@Alekhyareddy:~/OSSP$ |

```

Terminal-2 :

```
alekhya@Alekhyareddy: ~/OS × + v
alekhya@Alekhyareddy:~$ cd ~/OSSP
alekhya@Alekhyareddy:~/OSSP$ ps -ef | grep task_signal_actions
alekhya      488      301  0 04:56 pts/0    00:00:00 ./task_signal_actions
alekhya      511      494  0 04:56 pts/2    00:00:00 grep --color=auto task_signal_actions
alekhya@Alekhyareddy:~/OSSP$ kill -SIGUSR1 488
alekhya@Alekhyareddy:~/OSSP$ kill -SIGTERM 488
alekhya@Alekhyareddy:~/OSSP$ kill -SIGINT 488
alekhya@Alekhyareddy:~/OSSP$ ps -p 488
  PID TTY          TIME CMD
  488 pts/0    00:00:00 task_signal_act
alekhya@Alekhyareddy:~/OSSP$ ps -o pid,ppid,state,cmd -p 4525
  PID  PPID  S  CMD
alekhya@Alekhyareddy:~/OSSP$ kill -9 488
alekhya@Alekhyareddy:~/OSSP$ ps -p 488
  PID TTY          TIME CMD
alekhya@Alekhyareddy:~/OSSP$ |
```

2 . Capture SIGINT, Forward Signals, Protect Shell Process, Terminate Foreground Jobs, Update Job States, Test Interrupt Handling

Aim : This task demonstrates how a shell handles Ctrl+C (SIGINT) without terminating the shell itself, while forwarding the signal to the foreground job.

Program :

```
GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>

pid_t foreground_job = -1;
void handle_sigint(int sig)
{
    (void)sig;
    if (foreground_job > 0) {
        printf("\nShell: Forwarding SIGINT to foreground job (PID %d)\n",
            foreground_job);
        kill(foreground_job, SIGINT);
    } else {
        printf("\nShell: SIGINT received, but no foreground job is running.\n");
    }
}

int main()
{
    struct sigaction sa;
    pid_t pid;
    int status;
    sa.sa_handler = handle_sigint;
    sigemptyset(&sa.sa_mask);
    sa.sa_flags = 0;
    if (sigaction(SIGINT, &sa, NULL) == -1) {
        perror("sigaction");
        return 1;
    }
    printf("Shell Process Started\n");
    printf("Shell PID: %d\n", getpid());
    printf("Press Ctrl+C to interrupt the foreground job.\n\n");
    pid = fork();
    if (pid < 0) {
        perror("fork");
        return 1;
    }
    Help Write Out Where Is Cut Execute
```

```
pid = fork();

if (pid < 0) {
    perror("fork");
    return 1;
}

if (pid == 0) {
    signal(SIGINT, SIG_DFL);

    printf("Foreground job started.\n");
    printf("Job PID: %d\n", getpid());
    printf("Job State: RUNNING\n");
    printf("Waiting for SIGINT...\n");

    while (1) {
        sleep(1);
    }
}

foreground_job = pid;

printf("Shell: Foreground job PID = %d\n", foreground_job);
printf("Shell: Job State = RUNNING\n");
printf("Shell is waiting for the foreground job...\n\n");

waitpid(foreground_job, &status, 0);

if (WIFSIGNALED(status)) {
    printf("\nShell: Foreground job terminated by signal %d\n",
        WTERMSIG(status));
    printf("Shell: Job State = TERMINATED\n");
}
else if (WIFEXITED(status)) {
    printf("\nShell: Foreground job exited normally.\n");
    printf("Shell: Job State = COMPLETED\n");
}

foreground_job = -1;

printf("Shell Process is still running.\n");
printf("Shell Process Protected Successfully.\n");

return 0;
}
```

Output :

```
alekhya@Alekhyaireddy: ~/OS  X + v
alekhya@Alekhyaireddy:~$ cd ~/OSSP
alekhya@Alekhyaireddy:~/OSSP$ nano task_sigint_forward.c
alekhya@Alekhyaireddy:~/OSSP$ gcc -Wall -Wextra task_sigint_forward.c -o task_sigint_forward
alekhya@Alekhyaireddy:~/OSSP$ ./task_sigint_forward
Shell Process Started
Shell PID: 708
Press Ctrl+C to interrupt the foreground job.

Shell: Foreground job PID = 709
Shell: Job State = RUNNING
Shell is waiting for the foreground job...

Foreground job started.
Job PID: 709
Job State: RUNNING
Waiting for SIGINT...
^C
Shell: Forwarding SIGINT to foreground job (PID 709)

Shell: Foreground job exited normally.
Shell: Job State = COMPLETED
Shell Process is still running.
Shell Process Protected Successfully.
alekhya@Alekhyaireddy:~/OSSP$ |
```